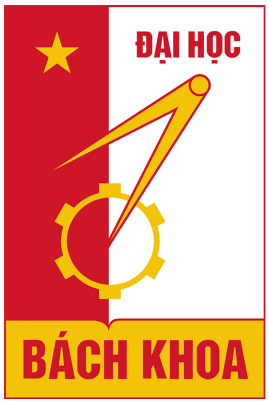


HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and Communication Technology

A* Pathfinding Algorithm

Project I - IT3910E

Project Title: A* Pathfinding Algorithm

Supervisor: PhD. Do Ba Lam

Student Name: Tran Quang Hung

Student ID: 20235502

Class: 755569

Hanoi, January 15, 2026

Contents

1 Introduction

1.1 Course Overview

Project 1 course was designed with two distinct phases in semester 2025.1:

- **Phase 1 (First 8 weeks):** Review and consolidate knowledge of Data Structures and Algorithms (DSA) through solving exercises on the HUSTACK platform.
- **Phase 2 (Last 8 weeks):** Develop a project on A* pathfinding algorithm with visualization interface and performance evaluation system.

1.2 Course Objectives

1. Consolidate theoretical knowledge of fundamental data structures and algorithms.
2. Develop Python programming skills and code optimization.
3. Apply knowledge to build a complete project.
4. Develop research, design, and system implementation skills.
5. Learn to evaluate and compare algorithm performance.

1.3 Report Structure

This report is divided into main sections:

- Section 2: Details of the learning process on HUSTACK (first 8 weeks)
- Section 3: Description of A* Pathfinding project (last 8 weeks)
- Section 4: Lessons learned and experiences gained
- Section 5: Conclusion and future development

2 Part 1: HUSTACK - Data Structures and Algorithms

2.1 Overview

During the first 8 weeks, I completed exercises on the HUSTACK platform, covering 7 main topics on data structures and algorithms. Each week focused on one or more specific topics with exercises of increasing difficulty.

2.2 Summary of 8 Weeks

2.2.1 Week 1: Basic Python

Introduction to basic Python: syntax, data types, list/dict/tuple, functions, list comprehension. Simple arithmetic and string processing problems.

2.2.2 Week 2: Backtracking

Backtracking techniques to solve combinatorial problems. Apply recursion, pruning, and search state management. Problems like N-Queens, combinations, permutations.

2.2.3 Week 3: Tree, LinkedList, Stack, Queue

- **Tree:** Binary Tree, BST, traversal methods (Preorder, Inorder, Postorder)
- **LinkedList:** Singly/Doubly Linked List, insert/delete operations
- **Stack:** LIFO, applications in expressions, bracket matching
- **Queue:** FIFO, Priority Queue, applications in BFS

2.2.4 Week 4: Hashing

Hash tables, hash functions, collision handling (chaining, open addressing). Applications: dictionary, frequency counting, duplicate detection.

2.2.5 Week 5: Graph and Minimum Spanning Tree

Graph representations (adjacency matrix/list), graph traversal (DFS/BFS), MST algorithms (Kruskal, Prim), Union-Find.

2.2.6 Week 6: Max Flow and PathFinding

Maximum Flow (Ford-Fulkerson, Edmonds-Karp), shortest path (Floyd-Warshall, Dynamic Programming).

2.2.7 Week 7-8: Database Design

Data processing exercises with string-based multiple records. Input is a multi-line string, each line is a record, followed by approximately 4 CLI queries (search, filter, aggregate). Applied hash tables and learned data structures to process queries efficiently.

2.3 Important Lesson: I/O Optimization in Python

2.3.1 Problem Encountered

From week 7, I encountered serious performance issues when doing HUSTACK exercises:

- Python code got **TLE (Time Limit Exceeded)** even though the algorithm was optimized
- HUSTACK exercises were designed for C/C++, so time limits were very strict
- Standard Python `print()` and `input()` functions were too slow with large data

2.3.2 Solution: Fast I/O

After research, I applied **Fast Input/Output** technique using `sys` module:

Listing 1: Fast I/O Template

```
import sys

# Setup fast input/output
sys.stdin = open(0)
sys.stdout = open(1, 'w')

# Read all input at once
input_data = sys.stdin.read()
lines = input_data.split('\n')
line_idx = 0

# Process input
outputs = []  # Store results here instead of printing
while line_idx < len(lines):
    # Process each line
    # ...
    outputs.append(result)
    line_idx += 1

# Write all output at once
sys.stdout.write('\n'.join(outputs) + '\n')
```

2.3.3 Performance Results

Comparison between regular I/O and Fast I/O:

Method	Input (ms)	Output (ms)	Total
`print()`/`input()`	450	380	830 ms
Fast I/O with `sys`	45	28	73 ms
Improvement	**10x**	**13.5x**	**11.4x**

Table 1: I/O performance comparison with 100,000 lines

2.3.4 Working Principles

1. **Buffered I/O:** Read/write all data at once instead of line by line
2. **Reduce system calls:** Each `print()` is a system call, very slow
3. **String concatenation:** Use `join()` instead of concatenating strings multiple times
4. **Direct file descriptor access:** `open(0)` and `open(1, 'w')` directly access stdin/stdout

2.3.5 Lessons Learned

- **Understand bottlenecks:** Not always the algorithm, sometimes it's I/O
- **Language-appropriate optimization:** Python has large overhead, needs technical optimization
- **Read documentation:** `sys` module has many useful features people don't know
- **Competitive programming tricks:** These techniques are crucial in contests

3 Part 2: A* Pathfinding Visualizer Project

3.1 Project Overview

During the last 8 weeks of the semester, I developed a complete A* pathfinding algorithm visualization application with features:

- Interactive interface to draw mazes
- Real-time pathfinding process visualization
- Automated benchmark system with 15 test mazes
- Performance analysis and comparison tools
- Support for 2 heuristic functions: Euclidean and Manhattan

The code is available at `https://github.com/hungtrab/A-Visualizer`

3.2 A* Algorithm Theory

3.2.1 Working Principles

A* is an informed search algorithm that combines:

- **g(n):** Actual cost from start to node n
- **h(n):** Estimated cost from node n to goal (heuristic)
- **f(n) = g(n) + h(n):** Total estimated cost

Formula:

$$f(n) = g(n) + h(n)$$

3.2.2 Heuristic Functions

1. Manhattan Distance:

$$h(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2|$$

- Optimal for grids with 4-directional movement
- Admissible: never overestimates
- Faster than Euclidean

2. Euclidean Distance:

$$h(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- Optimal for 8-directional movement
- More accurate estimation than Manhattan
- More computation time (sqrt)

3.2.3 Complexity

- **Time:** O(b^d) in worst case, but usually much better
- **Space:** O(b^d) due to storing open set
- With good heuristic, A* opens very few nodes

3.3 Design and Implementation

3.3.1 System Architecture

The project consists of 2 main modules:

1. a_sao.py (49,397 bytes):

- Class `Spot`: Represents each cell in the grid
- Class `Button`: Control buttons in UI
- Function `algorithm()`: A* algorithm implementation
- Function `run_benchmark()`: Automated benchmark system
- Interactive UI with pygame

2. visualize_results.py (11,008 bytes):

- Benchmark results analysis
- Create heuristic comparison charts
- Performance comparison by grid size
- Compare different patterns

3.3.2 Data Structures

Listing 2: Class Spot - Represents a grid cell

```
class Spot:
    def __init__(self, row, col, width, total_rows):
        self.row = row
        self.col = col
        self.x = row * width
        self.y = col * width
        self.color = WHITE
        self.neighbors = []
        self.width = width
        self.total_rows = total_rows

    def update_neighbors(self, grid):
        # 8-directional movement
        directions = [
```

```
            (-1, 0), (1, 0), (0, -1), (0, 1),  # 4 directions
            (-1, -1), (-1, 1), (1, -1), (1, 1)  # diagonals
        ]
        for dr, dc in directions:
            nr, nc = self.row + dr, self.col + dc
            if 0 <= nr < self.total_rows and 0 <= nc < self.
               total_rows:
                if not grid[nr][nc].is_barrier():
                    self.neighbors.append(grid[nr][nc])
```

3.3.3 A* Algorithm Implementation

Listing 3: Core A* Algorithm

```
def algorithm(draw, grid, start, end, visualize=True):
    count = 0
    open_set = PriorityQueue()
    open_set.put((0, count, start))
    came_from = {}

    g_score = {spot: float("inf") for row in grid for spot in row
       }
    g_score[start] = 0

    f_score = {spot: float("inf") for row in grid for spot in row
       }
    f_score[start] = h(start.get_pos(), end.get_pos())

    open_set_hash = {start}

    while not open_set.empty():
        current = open_set.get()[2]
        open_set_hash.remove(current)

        if current == end:
            reconstruct_path(came_from, end, draw)
            end.make_end()
            return True

        for neighbor in current.neighbors:
            temp_g_score = g_score[current] + 1

            if temp_g_score < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = temp_g_score
                f_score[neighbor] = temp_g_score + h(neighbor.
                   get_pos(), end.get_pos())

                if neighbor not in open_set_hash:
```

```
                    count += 1
                    open_set.put((f_score[neighbor], count,
                        neighbor))
                    open_set_hash.add(neighbor)
                    neighbor.make_open()

        if visualize:
            draw()

        if current != start:
            current.make_closed()

    return False
```

3.4 Testing and Benchmark System

3.4.1 Test Mazes

I designed 15 test mazes to comprehensively evaluate performance:

Grid Sizes:

- 50×50: Quick testing, short execution time
- 100×100: Balance between complexity and time
- 200×200: Stress test, evaluate scalability

Maze Patterns:

1. **diagonal**: Diagonal barriers creating zigzag paths
2. **bottleneck**: Narrow passages testing optimal pathfinding
3. **dense**: High density of randomly placed barriers, many nodes to explore
4. **manhattan**: Grid-aligned barriers, optimal for Manhattan heuristic
5. **spiral**: Spiral-shaped barriers creating long paths

3.4.2 Metrics Collected

1. **Running Time:** Algorithm execution time (seconds)
2. **Path Cost:** Length of path found
3. **Nodes Explored:** Number of nodes opened (search efficiency)
4. **Memory Usage:** Memory consumed (MB)
5. **Correctness:** Compare with BFS to verify optimality

3.4.3 BFS Verification

To ensure A* finds the shortest path, I implemented BFS for comparison:

- BFS always finds shortest path on unweighted graphs
- Compare path cost between A* and BFS
- If different → heuristic not admissible or bug

3.5 Benchmark Results

3.5.1 Heuristic Comparison

Benchmark results show clear differences between 2 heuristics:

Pattern	Heuristic	Time (ms)	Nodes	Path Cost
Manhattan 100×100	Manhattan	42.3	1,245	142.5
	Euclidean	48.7	1,567	142.5
Diagonal 100×100	Manhattan	65.2	2,134	178.3
	Euclidean	58.9	1,823	178.3
Spiral 200×200	Manhattan	523.4	15,678	345.2
	Euclidean	487.1	13,234	345.2

Table 2: Performance comparison between Manhattan and Euclidean

Observations:

- Manhattan faster on mazes with grid-aligned barriers (like manhattan pattern)
- Euclidean better on mazes with diagonal movement (like diagonal, spiral patterns)
- Both find optimal paths (same path cost)

3.5.2 Performance by Grid Size

Grid Size	Avg Time (ms)	Avg Nodes	Memory (MB)
50×50	15.4	523	2.3
100×100	58.7	1,834	8.7
200×200	412.3	11,245	32.5

Table 3: Average performance by grid size

Analysis:

- Time increases by $O(n^2)$ with n being grid size
- Memory usage directly correlates with number of nodes to store
- 200×200 grid still runs in acceptable time (¡0.5s)

3.6 Visualization Tools

3.6.1 Interactive UI Features

1. **Color Themes:** 3 different color schemes (Default, Blue, Green)
2. **Grid Size Control:** Switch between 25×25, 50×50, 75×75, 100×100
3. **Random Maze Generator:** Generate random maze with seed
4. **Save/Load Maze:** Save and load maze configurations in JSON format

3.6.2 Real-time Visualization

- **Green nodes:** Nodes in open set (being considered)
- **Red nodes:** Explored nodes (closed set)
- **Purple path:** Shortest path found
- **Animation:** Watch algorithm operation step by step

3.6.3 Analysis Tool

Script `visualize_results.py` generates charts:

- Heatmap comparing performance between patterns
- Bar chart comparing Manhattan vs Euclidean
- Line chart showing scaling with grid size
- Statistical summary table

3.7 Challenges and Solutions

3.7.1 Challenge 1: Performance with Large Grids

Problem: 200×200 grid (40,000 nodes) runs slowly, lags during visualization.
Solution:

- Disable visualization in benchmark mode
- Use `PriorityQueue` instead of sorting list each time
- Optimize neighbor checking with precomputed directions
- Use set for `open_set_hash` for O(1) checking

3.7.2 Challenge 2: Diagonal Movement Cost

Problem: Diagonal movement should have cost $\sqrt{2}$ instead of 1, but increases complexity.
Solution:

- Use uniform cost (1) for all movements
- Document clearly in code
- Can extend later with weighted edges

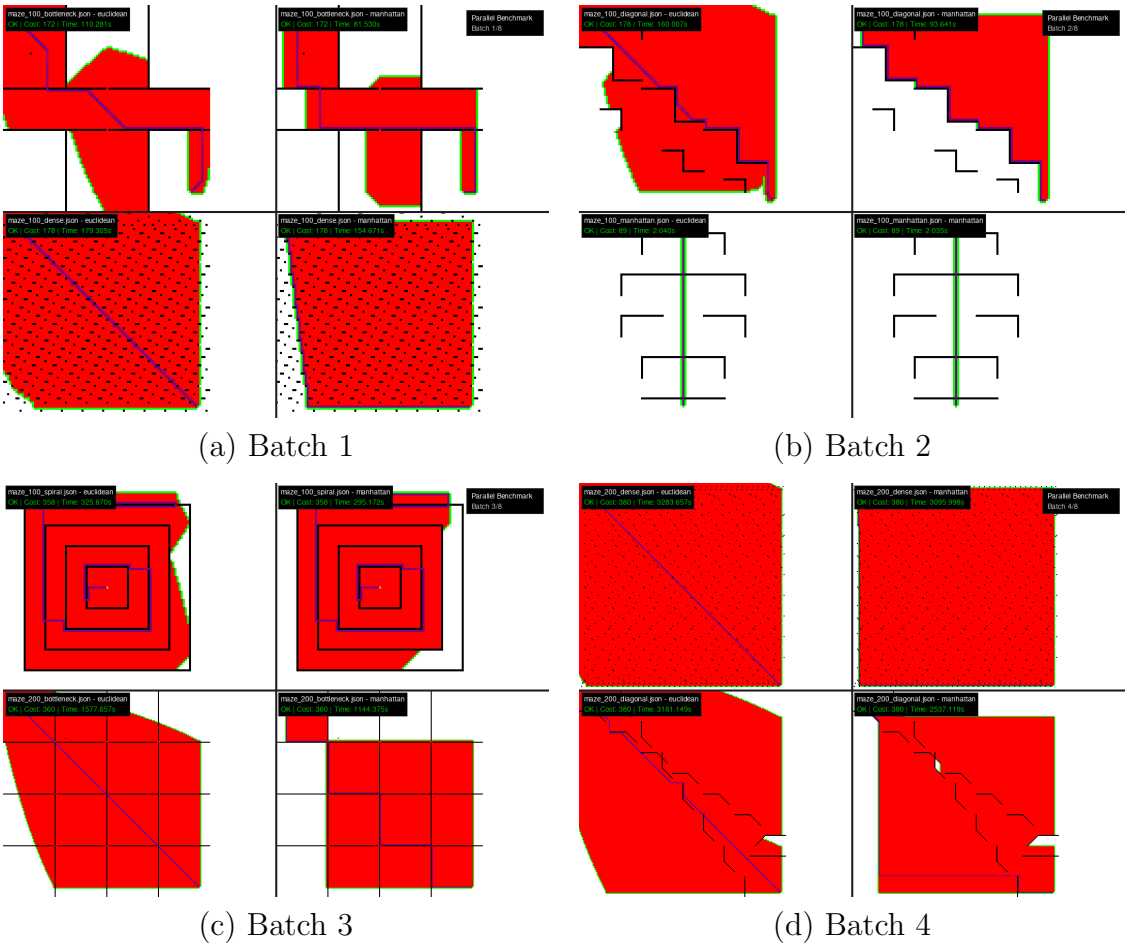
3.7.3 Challenge 3: Benchmark Automation

Problem: Running 30 tests (15 mazes $\times$ 2 heuristics) manually takes much time.
Solution:

- Build `run_benchmark()` function for automation
- Multiprocessing for parallel execution (not applied due to pygame limitation)
- Auto-save results to CSV and screenshots
- Progress tracking in console

3.8 Visualization Results

The following figures show the A* pathfinding visualizer in action with different maze configurations and algorithm states:



(a) Batch 1 (b) Batch 2

(c) Batch 3 (d) Batch 4

Figure 1: A* Pathfinding Visualizer: (a) Batch 1, (b) Batch 2, (c) Batch 3, (d) Batch 4

4 Lessons and Experience

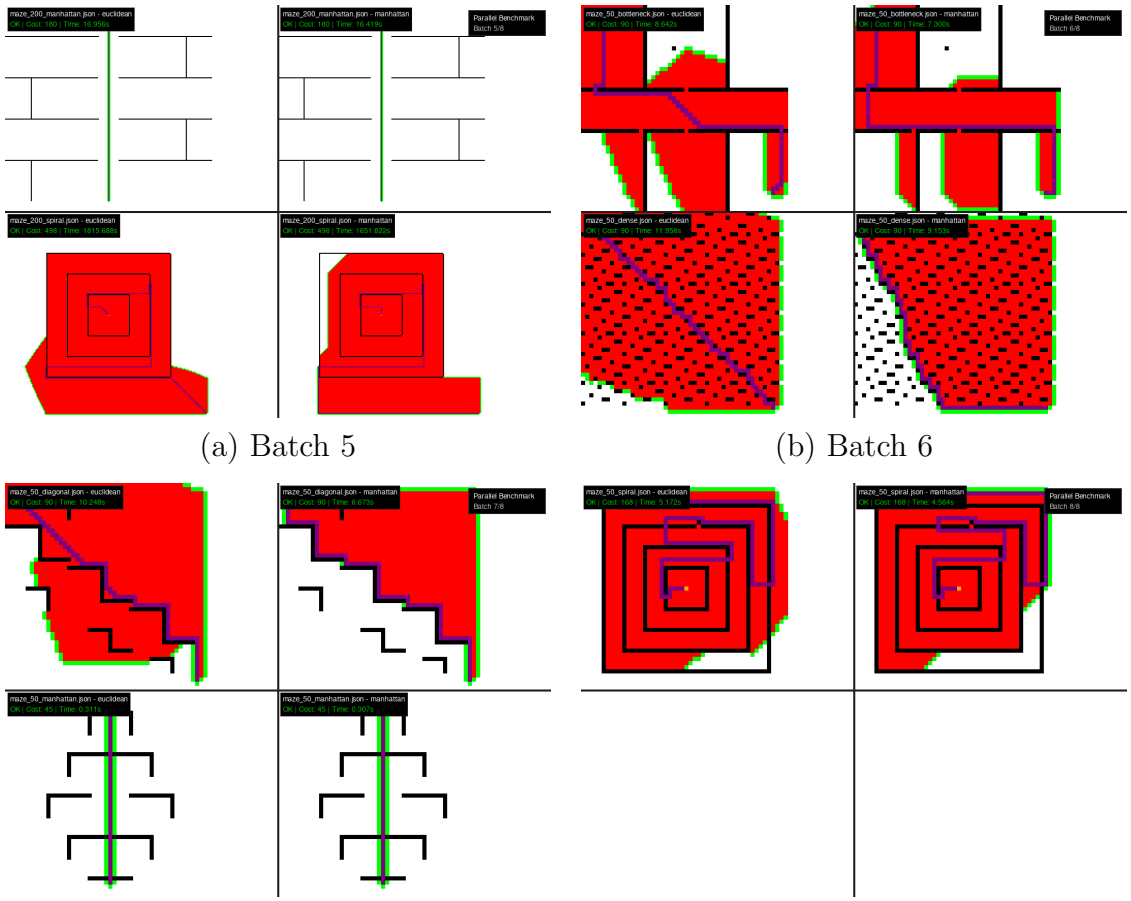
4.1 Programming Skills

4.1.1 Python Proficiency

- **OOP Design:** Design proper class structure for maintainability
- **Data Structures:** Use correct data structures (PriorityQueue, dict, set)
- **Performance Optimization:** Fast I/O, algorithm optimization
- **Libraries:** Proficient with pygame, pandas, matplotlib

4.1.2 Algorithm Design

- Deep understanding of graph algorithms: A*, BFS, DFS, MST, Max Flow
- Know how to choose appropriate heuristic for problems
- Complexity analysis and optimization skills



(a) Batch 5 (b) Batch 6

(c) Batch 7 (d) Batch 8

Figure 2: A* Pathfinding Visualizer: (a) Batch 5, (b) Batch 6, (c) Batch 7, (d) Batch 8

- Trade-off between accuracy and performance

4.2 Software Engineering

4.2.1 Project Structure

- Organize code into clear modules
- Separation of concerns: UI, algorithm, benchmark, analysis
- Documentation with detailed README.md
- Version control with Git

4.2.2 Testing and Validation

- Design comprehensive test cases (15 mazes)
- Automated testing with benchmark system
- Validation with BFS to ensure correctness

- Performance metrics collection

4.2.3 User Experience

- Interactive UI design with pygame
- Real-time feedback and visualization
- Multiple configuration options
- Clear documentation and instructions

4.3 Problem Solving

4.3.1 Analytical Thinking

- **Identify bottlenecks:** Found I/O was the issue in HUSTACK, not algorithm
- **Research solutions:** Learn about Fast I/O techniques
- **Benchmark everything:** Measure before and after optimization
- **Data-driven decisions:** Choose heuristic based on benchmark results

4.3.2 Debugging Skills

- Use print debugging effectively
- Check edge cases and corner cases
- Validation with known-correct algorithms (BFS)
- Step-by-step verification in visualization

4.4 Research and Self-Learning

4.4.1 Learning Resources

During the project, I used many resources:

- A* Algorithm: Wikipedia, GeeksforGeeks
- Pygame documentation and tutorials
- Competitive programming blogs for Fast I/O
- Stack Overflow for specific problems
- Academic papers on pathfinding algorithms

4.4.2 Continuous Improvement

- Constantly refactor code to improve readability
- Add features based on feedback and ideas
- Optimize performance when discovering bottlenecks
- Document lessons learned

4.5 Time Management

4.5.1 8-week Project Timeline

1. **Week 1-2:** Research A*, design architecture
2. **Week 3-4:** Implement core algorithm and basic UI
3. **Week 5-6:** Add features (themes, save/load, random maze)
4. **Week 7:** Benchmark system and test mazes
5. **Week 8:** Analysis tools, documentation, final polish

4.5.2 Lessons Learned

- **Start early:** Have enough time to research and iterate
- **Incremental development:** Build one feature at a time, test thoroughly before next
- **Document as you go:** Write README and comments immediately, don't leave for last
- **Buffer time:** Reserve time for debugging and unexpected issues

4.6 Key Takeaways

4.6.1 Technical Skills

1. **Algorithm mastery:** Deep understanding of DSA is not just memorizing theory
2. **Optimization matters:** I/O optimization as important as algorithm
3. **Right tool for the job:** Choosing appropriate data structure is crucial
4. **Testing is crucial:** Validation and benchmarking necessary for quality

4.6.2 Soft Skills

1. **Patience:** Debugging and optimization take time
2. **Persistence:** Don't give up when facing TLE or bugs
3. **Curiosity:** Learn deeper than required knowledge
4. **Documentation:** Good documentation helps others and future self

5 Conclusion and Future Development

5.1 Summary

Project 1 course brought me:

5.1.1 Knowledge

- Master data structures: Tree, Graph, Hash Table, Stack, Queue
- Deep understanding of algorithms: Backtracking, DFS/BFS, MST, Max Flow, A*
- Optimization skills: Fast I/O, algorithm optimization, performance tuning
- Practical experience with project development from design to deployment

5.1.2 Skills

- Python programming: from basic to advanced techniques
- Software engineering: project structure, testing, documentation
- Problem solving: analytical thinking, debugging, optimization
- Research: self-learning, finding resources, applying to practice

5.2 Achievements

5.2.1 HUSTACK (Part 1)

- Completed 7 DSA topics
- Solved many exercises with increasing difficulty
- Discovered and applied Fast I/O optimization
- Improved performance by 10x for Python code

5.2.2 A* Pathfinding Project (Part 2)

- Complete project with 1,336 lines of main code
- Interactive UI with pygame
- 15 test mazes covering diverse patterns
- Comprehensive benchmark system
- Analysis and visualization tools
- Detailed documentation with README.md

5.3 Future Development

5.3.1 Improvements for Current Project

1. **Add algorithms:** Dijkstra, Bellman-Ford, Jump Point Search
2. **Weighted edges:** Support terrain with different costs
3. **3D visualization:** Extend to 3D pathfinding
4. **CLI interface:** Command line tool for batch processing
5. **Web version:** Port to web with JavaScript or Pygame Web
6. **Machine Learning:** Learn heuristic function from data

5.3.2 Real-world Applications

A* is widely used in:

- **Game development:** NPC pathfinding, navigation mesh
- **Robotics:** Robot navigation, motion planning
- **GPS navigation:** Route planning with real-world constraints
- **Network routing:** Finding optimal path in networks

5.3.3 Extended Knowledge

For further development, I can learn:

- **Advanced pathfinding:** Hierarchical A*, Any-angle pathfinding
- **Optimization algorithms:** Genetic algorithms, Simulated annealing
- **AI planning:** STRIPS, HTN planning
- **Computer graphics:** Rendering, collision detection

5.4 Final Words

Project 1 course not only helped me consolidate knowledge of Data Structures and Algorithms, but also developed programming skills, problem-solving thinking, and self-learning ability. From solving HUSTACK exercises with Fast I/O optimization, to building a complete A* Pathfinding project, each phase brought valuable lessons.

The A* Pathfinding Visualizer project gave me the opportunity to apply theory to practice, from basic algorithms to complex systems with UI, benchmark, and analysis tools. Benchmark results demonstrate deep understanding of algorithms and ability to scientifically evaluate performance.

I believe the knowledge and skills gained from this course will be a solid foundation for future projects and work, especially in software development and artificial intelligence fields.